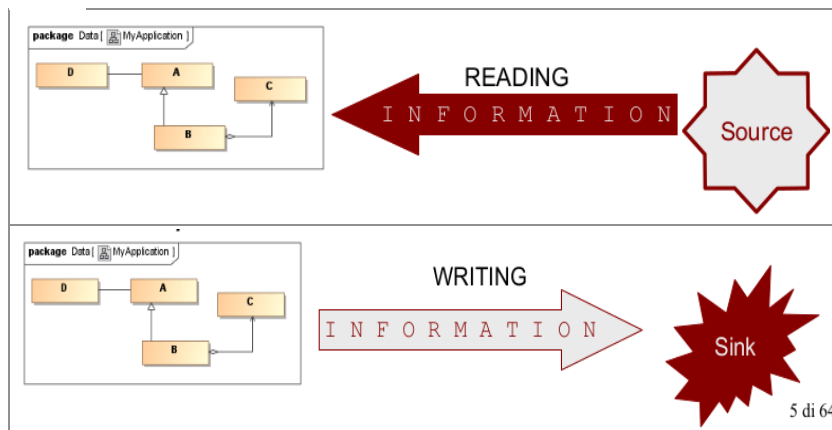


Lezione 11 4/12/23 I/O

mercoledì 6 dicembre 2023 17:01

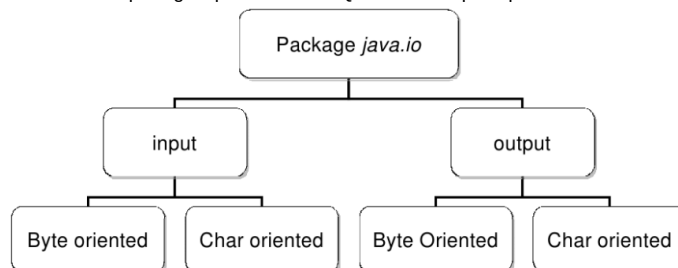
Concentriamoci ora sulla gestione dell'I/O in java : per prima cosa ci devono essere dispositivi omogenei (files, console, dispositivi in rete, tastiera ecc) , formati di dato omogenei (testo audio, binario ecc) , modalità di accesso (sequenziale, bufferizzato , ad accesso casuale) e modalità di interazione (bit ,byte ,word ecc). Quindi in java per soddisfare queste richieste si usano una vasta gamma di librerie e classi , le quali al loro interno incapsulano le operazioni per gestire un tipo di dato ,un tipo di accesso , una particolare tipologia di dispositivi . Vi è possibile cooperare tra tutte queste librerie e classi per ottenere la completa gestione dell'I/O , magari specializzando le classi e di conseguenza i comportamenti (metodi). Abbiamo detto come gestirlo , ma in realtà ci serve un mezzo per fare ciò : **lo stream** : buffer di memoria che modella il flusso dati di un qualunque dispositivo capace di produrre o ricevere dati . Lo stream di fatto è un concetto "astratto" , in quanto non considera realmente i dettagli della gestione della trasmissione/ricezione dei dati da un qualunque dispositivo : di solito è sequenziale ma a volte può essere anche ad accesso casuale . Quindi in generale per permettere la trasmissione/ricezione di dati da/verso sorgente/destinazione , si deve aprire uno stream tra questi due o più dispositivi , permettendo così la scrittura/lettura in modo sequenziale delle info.



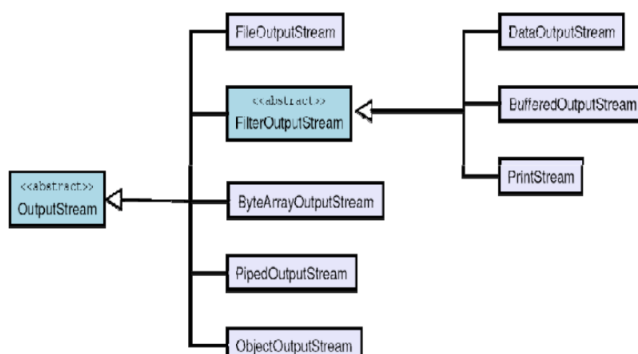
Quindi il processo per istanziare questa comunicazione (creando ed usando lo stream) avviene seguendo questi passi :

- | | |
|--------------------------------------|--------------------------------------|
| 1. open (stream) | 1. open (stream) |
| 2. while (more information ?) | 2. while (more information ?) |
| 3. read (information) | 3. write (information) |
| 4. close (stream) | 4. close (stream) |

Come già detto java usa varietà di librerie e classi per permettere questa comunicazione attraverso lo stream , ed in dettaglio usa la classe **java.io.*** , la quale al suo interno definisce classi ed interfacce per ogni tipo di stream . Queste classi principalmente sono suddivise così :

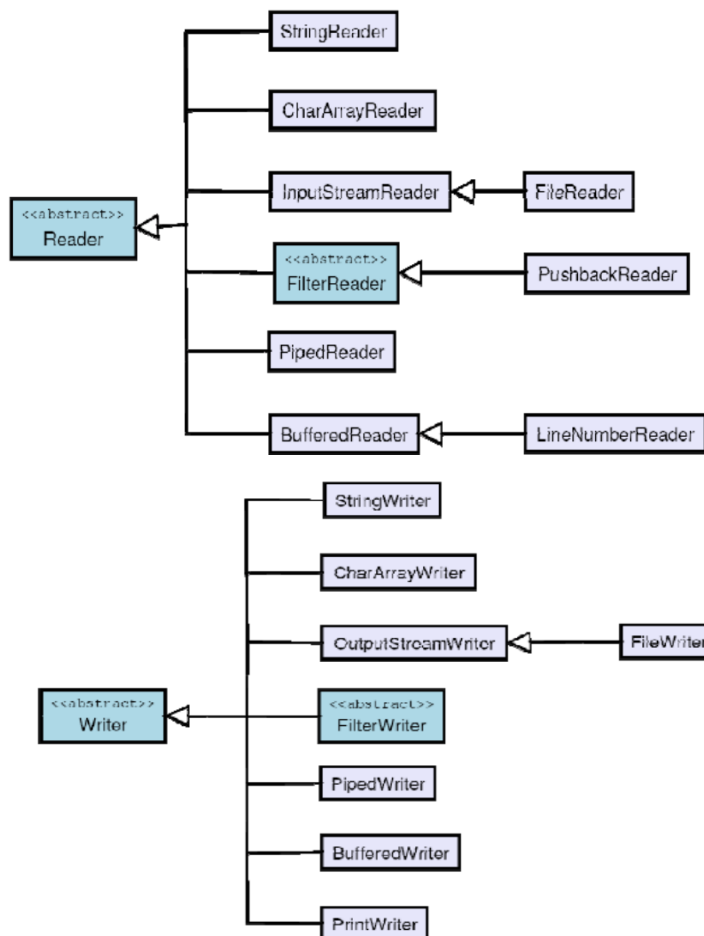


Dove nel Byte oriented l'unità dati che viaggia è di tipo byte , e si ha i/o binario , si usano InputStream per input mentre OutputStream per output ; analogamente al flusso binario c'è il flusso testuale (codifica e flusso ascii) e le librerie usate sono Reader (lettura) e Writer (scrittura) . Tornando alla classificazione tra input ed output , in realtà queste classi sono super classi di altre classi :





Finora abbiamo detto che per far comunicare due dispositivi serve lo stream (vero, ma non basta): serve un oggetto che crea fisicamente il collegamento con il file fisico ed un oggetto che gestisce lo stream, inviandoci dati sopra: istanza di sotto classe di **OutputStream**. Questo collegamento viene fatto dalla classe **FILE**: la quale classe fornisce una rappresentazione astratta ed indipendente dal sistema dei pathname gerarchici: **rappresenta solo il nome di un particolare file o nome di gruppi di file di una directory e consente di creare un collegamento con il file fisico**. Vediamo ora le librerie usate per gestire stream testuali:



Vediamo ora degli esempi:

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4
5  public class InputBufferizzatoDaFile {
6      public static void main(String args[]) throws IOException {
7          //apro il file come stream di caratteri -> FileReader
8          //uso lettori bufferizzati per aumentare prestazione -> bufferedReader
9          BufferedReader in=new BufferedReader( new FileReader( fileName: "file.txt"));
10         String s= "";
11         StringBuilder s2= new StringBuilder();
12         //leggo ogni linea del file
13         while((s=in.readLine())!=null)
14         {
15             s2.append(s).append("\n");
16         }
17         //stream associato al file viene chiuso
18         in.close();
19     }
20 }
  
```

```

1  import java.io.IOException;
2  import java.io.StringReader;
3
4  ▶ public class InputDiCaratteriDaMemoria {
5  ▶      public static void main(String []args) throws IOException {
6          String s2="this is foo!!";
7          //creo un lettore di flussi di catatteri a partire da s2
8          StringReader in=new StringReader(s2);
9          int c;
10         //i caratteri nel flusso vengono letti
11         //sequenzialmente con il metodo read
12         //quando tutti caratteri sono
13         //stati letti l'iterazione si interrompe
14         while ((c=in.read())!= -1)
15         {
16             //il carattere letto ha una rappresentazione
17             //intera; va convertito
18             System.out.println(" char : " + ((char)c));
19         }
20     }
21     /*
22     char :t
23     char :h
24     char :i
25     char :s
26     char :
27     char :i
28     char :s
29     char :
30     char :f
31     char :o
32     char :o
33     char :!
34     char :!
35     */
36 }

```

```

1  import java.io.ByteArrayInputStream;
2  import java.io.DataInputStream;
3  import java.io.EOFException;
4  import java.io.IOException;
5
6  ▶ public class InputByteDaMemoria{
7  ▶      public static void main(String[] args) throws IOException {
8          String s2 = "this is foo!!";
9          try {
10             //getBytes converte la stringa in una
11             //sequenza di byte utilizzando la
12             //rappresentazione specifica di Java
13             // ByteArrayInputStream viene creato un lettore di flussi di byte ...
14             //con DataInputStream consente di gestire tutti i tipi di dato primitivi di Java riuscendo
15             //ad avere un codice più flessibile
16             DataInputStream in3 = new DataInputStream( new ByteArrayInputStream(s2.getBytes()));
17             while(true) {
18                 //viene letto un byte per volta dallo stream
19                 System.out.print((char) in3.readByte());
20             }
21         } catch (EOFException e){
22             System.err.print(" End of stream");
23         }
24     }
25     // this is foo!! End of stream
26 }

```

```

1 import java.io.*;
2 public class OutputCharacterToFile {
3     public static void main(String args[])
4     {
5         String s2 = "this is foo!";
6         try {
7             BufferedReader in3 = new BufferedReader( new StringReader(s2));
8             //FileWriter -> apertura file come stream di output a caratteri
9             //BufferedWriter -> buffering per migliorare le performance di accesso
10            //PrintWriter -> gestisce lo stream attraverso una classe che supporta la stampa di vari tipi basici formattandoli come testo
11            PrintWriter out1 = new PrintWriter( new BufferedWriter(new FileWriter( filename: "file.txt")));
12            int lineCount = 0;
13            String g;
14            //lo stream di input è letto una linea alla volta. l'iterazione termina al raggiungimento della fine del flusso
15            while((g = in3.readLine()) != null){
16                lineCount++;
17                out1.println(lineCount + ": " + g);
18            }
19            //gli stream bufferizzati non garantiscono che il loro contenuto venga effettivamente scritto nel file con l'invocazione dei metodi scrittura.
20            //close() forza la scrittura dei dati e chiude il file.
21            out1.println();
22            out1.close();
23        } catch (IOException e) {
24            System.err.print("end of stream");
25        }
26    }
27    //scrive sul file this is foo!
28 }

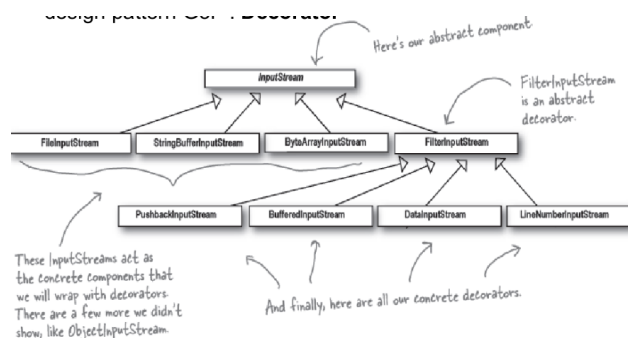
```

```

1 import java.io.*;
2 import java.nio.file.Files;
3 import java.nio.file.Paths;
4
5 public class InputOutputBytesToFile {
6     public static void main(String args[])
7     {
8         try {
9             DataOutputStream out2 = new DataOutputStream(
10                 new BufferedOutputStream(Files.newOutputStream(Paths.get( filename: "data.dat"))));
11             out2.writeDouble( @ 3.14159);
12
13             out2.writeUTF( @ "That was pi");
14             out2.writeDouble( @ 1.41413);
15             out2.writeUTF( @ "Square root of 2");
16             out2.close();
17             DataInputStream in5 = new DataInputStream(
18                 new BufferedInputStream( new FileInputStream( filename: "data.dat")));
19             // Must use DataInputStream for data:
20             System.out.println(in5.readDouble());
21             // Only readUTF() will properly recover the data
22             System.out.println(in5.readUTF());
23             // Read the following double and String:
24             System.out.println(in5.readDouble());
25             System.out.println(in5.readUTF());
26
27         } catch (IOException e) {
28             throw new RuntimeException(e);
29         }
30     }
31 }
32
33 3.14159
34 That was pi
35 1.41413
36 Square root of 2
37
38 }

```

Facciamo ora una breve riflessione : come già detto il package java.io.* è un'applicazione del GOF DECORATOR :



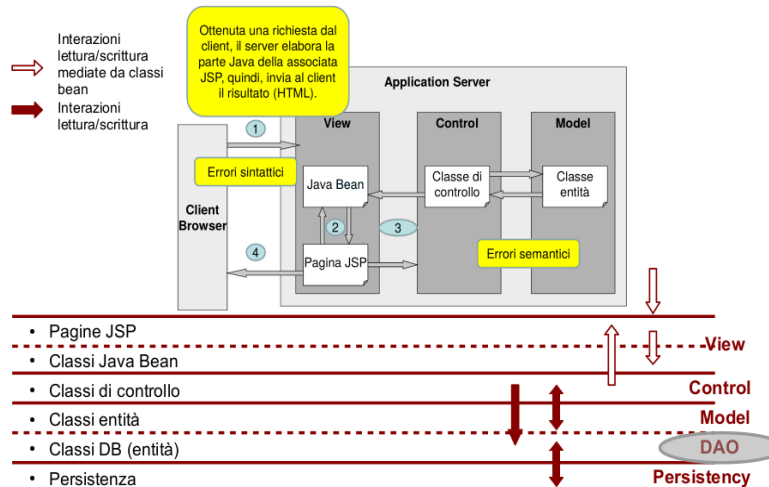
Sofferamoci ora sui file ad accesso casuale : file che non permettono solo l'accesso in maniera sequenziale, ma anche in modo randomico : questo accesso randomico viene consentito solamente mediante stream associati a file . Questa classe non fa parte di nessuna gerarchia delle classi fatte in precedenza : simile a DataInput/DataOutputStream , ma inoltre ha il metodo seek() : il quale permette di posizionarsi in un qualsiasi punto del file .

```

1 import java.io.IOException;
2 import java.io.RandomAccessFile;
3
4 public class LetturaScritturaSuFileRandom {
5     public static void main(String args[]) throws IOException {
6         RandomAccessFile rf = new RandomAccessFile( filename: "test.dat", mode: "rw");
7         for(int i = 0; i < 10; i++){
8             rf.writeDouble( @ i*1.414);
9             rf.close();
10
11             rf = new RandomAccessFile( filename: "test.dat", mode: "rw");
12             rf.seek( @ 50L);
13             rf.writeDouble( @ 47.8001);
14             rf.close();
15
16             rf = new RandomAccessFile( filename: "test.dat", mode: "r");
17             for(int i = 0; i < 10; i++){
18                 System.out.println("value " + i + ": " + rf.readDouble());
19             }
20         }
21     }
22 }
23
24 Value 0: 0.0
25 Value 1: 1.414
26 Value 2: 2.828
27 Value 3: 4.242
28 Value 4: 5.656
29 Value 5: 7.071
30 Value 6: 8.484
31 Value 7: 9.898
32 Value 8: 11.312
33 Value 9: 12.725
34
35 }

```

Una domanda sorge spontanea : quale è il ciclo di vita canonico di un oggetto?? Si perdono le informazioni riguardanti quell'oggetto dopo che viene "eliminato"?? La risposta è sì , ma comunque ci sono dei meccanismi per evitare ciò . Prima però andiamo a vedere il **ciclo di vita di un oggetto** : comincia con l'allocazione in memoria nella jvm , in seguito alla new ed invocando uno dei costruttori , per poi finire con la deallocazione esplicita (C) oppure garbage collector in java. Di solito il ciclo di vita è questo, ma esistono scenari nei quali si ha bisogno che lo stato dell'oggetto sopravviva alla distruzione dello stesso : o lo stato dell'oggetto continui ad esistere dopo la distruzione, oppure bisogna usare dei meccanismi che permettono di ripristinare lo stato dell'oggetto . Il che fa sì che lo stato dell'oggetto possa migrare tra più nodi di un'applicazione distribuita. Quindi nel contesto di applicazioni distribuite :



Ricordandoci che la persistenza degli oggetti viene fatta attraverso le classi bean. Come già detto , per sopprimere a questa mancanza di solito si usano due "modi" : **FILES** oppure **DBMS** : nei primi bisogna considerare tutti gli aspetti del tipo di dato : gestione delle convenzioni di memorizzazione e del formato dei dati , mentre nei secondi (non adatti a sistemi limitati) bisogna fare attenzione al mismatch tra oggetto O.O e quello nel DAO. Andiamo a vedere ora in dettaglio come fare "persistenza dei dati" (senza usare il DB) : due possibili approcci : **serializzazione e deserializzazione**. Il primo consente di salvare lo stato interno di un oggetto (istanza considerata + tutti i riferimenti in esso contenuto), mentre il secondo consente di ripristinare lo stato di un oggetto precedentemente salvato . Entrambe usano il concetto di stream (collegamento tra oggetti che permette flusso di dati) . In java.io ci sono 2 classi specifiche: *objectinputstream* e *objectoutputstream* . Come abilitare questo meccanismo?? **La classe in questione deve realizzare interfaccia SERIALIZABLE** : attenzione però: questa interfaccia è particolare in quanto non definisce alcuna operazione. Quindi questo meccanismo lavora su oggetti allocati nell'heap di memoria , quindi la jvm visto che non lavora su heap non mantiene questo riferimento , in quanto queste classi sono istanziate sempre e soltanto ad allocazione (inizializzate quindi con il valore statico della classe) . Visto che java non ha meccanismi per "salvare" le variabili statiche : ne serializzazione ne de serializzazione , quindi bisogna prevedere dei meccanismi per fare ciò. Riassumiamo ora tutti i tipi di dato per fare I/O:

Tipo di I/O	Classi (Char Oriented/Byte Oriented)	Descrizione
Memoria	<code>CharArrayReader</code> <code>CharArrayWriter</code> <code>ByteArrayInputStream</code> <code>ByteArrayOutputStream</code>	Questi stream sono utilizzati per leggere o scrivere su degli array già presenti in memoria
Memoria	<code>StringReader</code> <code>StringWriter</code> <code>StringBufferInputStream</code>	Stream per leggere o scrivere su delle stringhe utilizzando delle classi di tipo <code>StringBuffer</code>
Pipe	<code>PipedReader</code> <code>PipedWriter</code> <code>PipedInputStream</code> <code>PipedOutputStream</code>	Le pipe vengono usate per convogliare l'output di un processo nell'input di un altro
File	<code>FileReader</code> <code>FileWriter</code> <code>FileInputStream</code> <code>FileOutputStream</code>	Accesso sequenziale a file presenti nel filesystem

Tipo di I/O	Classi (Char Oriented/Byte Oriented)	Descrizione
Concatenazione	N/A <code>SequenceInputStream</code>	Concatena più input stream come se fossero uno solo
Object	N/A <code>ObjectInputStream</code> <code>ObjectOutputStream</code>	Consente di scrivere o leggere le rappresentazioni degli oggetti
Conversione dati	N/A <code>DataInputStream</code> <code>DataOutputStream</code>	Leggono o scrivono i tipi di dato primitivi in un formato indipendente dalla macchina
Stampa	<code>PrintWriter</code> <code>PrintStream</code>	Forniscono dei metodi convenienti per stampare le informazioni
Conteggio linee	<code>LineNumberReader</code> <code>LineNumberInputStream</code>	Tengono traccia del numero di linee di testo lette



Conversione

Tipo di I/O	Classi (Char Oriented/Byte Oriented)	Descrizione
Buffering	BufferedReader BufferedWriter BufferedInputStream BufferedOutputStream	Provvedono a fornire un buffer agli stream per rendere più efficienti le operazioni di input/output
Filtri	FilterReader FilterWriter FilterInputStream FilterOutputStream	Consentono di applicare dei filtri anche definiti dall'utente agli stream per processare automaticamente i dati letti o scritti
Conversione tra byte e caratteri	N/A InputStreamReader OutputStreamWriter	Convertono degli stream orientati ai byte in stream orientati a caratteri